

Indledende programmering Noter

Naming conventions

Type	Description	Example
Class	PascalCase	"myClass"
Method	camelCase	"myMethod"
Variable	camelCase	"myVariable"
Constant	SCREAMING_SNAKE_CASE	"THIS_IS_MY_CONSTANT"

```
// Note, for the class myClass, filename must be myClass.java
public class myClass {
    private static final String MY_CONSTANT = "I can't be changed";

    static void myMethod() {
        String myVariable = "Hello";
        System.out.println(myVariable + " " + MY_CONSTANT);
    }
}
```

Comments

```
// This is a one-line comment

/* This is a
multi-line
Comment */
```

Data types

Name	Description
Signed Integers	Integers within a specific range
byte	8-bit signed integer (-128 to 127)
short	16-bit signed integer (-32.768 to 32.767)
int	32-bit signed integer (-2^{31} to $2^{31}-1$)
long	64-bit signed integer (-2^{63} to $2^{63}-1$)
Floats	

Name	Description
float	Single-precision floating point
double	Double-precision floating point
Others	
boolean	True/False or 1/0 toggle
char	A single character (UTF16, no emojis)

Integer Operators

Navn	Symbol	Example
Addition	+	2+2=4
Subtraction	-	2-2=0
Multiplication	*	2*3=6
Division	/	3/2=1 3./2=1.5
Exponent	Math.pow(a,b)	Math.pow(2, 3)=8
Modulo	%	3%2=1

Order of operations

Just use plenty of parentheses to make sure everything works correctly.

1. Functions like `Math.pow`
2. Multiplication/Division
3. Addition/subtraction

Signed integer overflow

When going over or under the range, the number will wrap around.

```
byte a = 127;
System.out.println(a+1);
// Prints -128
```

Converting integers

Integers can be cast:

```
(int) 4.2 // returns 4
(double) 4 // returns 4.0
```

String

Not a primitive data type, therefore written as `String` not `string`

```
System.out.println("\t"); // tab character
System.out.println("\n"); // newline
System.out.println("\""); // quotation mark
System.out.println("\\"); // backslash
```

Converting types to String

```
// Integer, all these do the same
Integer.toString(integer);
integer + "";
String.valueOf(integer);

// Double
Double.toString(double);
double + "";
String.valueOf(double)
```

Comparing strings

```
String s = "Hello"
if (s == "Hello") {
    System.out.println("Does not work");
}
if (s.equals("Hello")) {
    System.out.println("This works!");
}
if (s.equalsIgnoreCase("hELLo")) {
    System.out.println("This also works!");
}
```

Char

a `char` is saved as a number, typically written with single quotes.

```
char myChar = 'a';
char backslash = '\\';
char quote = '\'';

System.out.println(myChar); // prints a
System.out.println(myChar+1); // prints 98
System.out.println(myChar+quote) // prints 136
```

To see what number a char becomes, see ASCII Table.

Floats

`double` and `float` are floating point, so the math is a bit weird. In general, it is a good idea to round them off if you need to compare them.

```
// Note, these are the same calculations in different orders
double foo = 0.01 + 0.05 + 0.10 + 0.25; // foo=0.41000000000000003
double bar = 0.25 + 0.10 + 0.05 + 0.01; // bar=0.41

System.out.println(0.1 + 0.2 == 0.3) // prints false, answer is
0.30000000000000004

// check if two numbers are within 0.001 of each other
if (Math.abs(foo - bar) < 0.001) {
    System.out.println("This works!");
}
```

Assignment operators

Simpler way of modifying an existing value

```
int x = 4;

x +=1; // x=5
x -=1; // x=4
x *=2 // x=8
x /=2 // x=4
```

Incrementing

Note: just always use the ones with symbols on the right.

```
x++ // adds 1
++x // adds 1
```

```
x-- // subtracts 1
--x // subtracts 1
```

Printing to console

```
System.out.print("Does not go to next line");
System.out.println("Prints and goes to next line");
System.out.println("""
This allows
for printing
multiple lines
easily
                it also trims leading spaces!
""");
```

printf

```
int myInt = 3
float myFloat = 4
System.out.printf("these are the values: %d %f \n", myInt, myFloat)
```

Note that printf does not automatically print a newline, therefore the included `\n`.

The commands for the formatting are written like:

```
d = int, f = double, s = String, b = boolean, etc.

- = left-aligned, + = positive numbers prefixed with +, [a space] = prefix
positive numbers with a space, 0 = pad numbers with zeroes on the left

numbers = width

.numbers = how many decimals to include

// Examples

%8d    = int, right aligned, in 8-space-wide-field
%12.4f = double, right aligned, in 12-space-wide-field, rounded to 4 decimals
%-6d   = int, left aligned, 6-space-wide-field
%-9s   = String, left aligned, 9-space-wide-field
```

If statements

```
if (test1 && test2) {
```

```

    // this requires both are true
} else if (test1 || test1) {
    // this requires at least one is true
} else {
    // this runs if no other runs
}

String input = "Hello";
int a = 0;

switch(input) {
    case "Hello":
        a=1;
        break;
    // Cases without break roll over to the next one down
    case "John":
    case "Josephine":
        a=4;
        break;
    // Default is used when no others match
    default:
        a=-1;
}

```

Ternary statements

Ternary operators allow for doing an if/else check on values. Ternary operators must return values.

```

// Assigns myValue = inputValue unless inputValue < 0, instead assigning
myValue = 0
Int myValue = inputValue < 0 ? 0 : inputValue;

// Prints "Hello World" if myValue is 5, prints "Hello Everyone!" otherwise.
System.out.println("Hello " + (myValue == 5 ? "World" : "Everyone!"))

```

Ternary operators can be stacked, but this should almost always be avoided

```

String x = y == 4 ? "A" : (y == 3 ? "B" : "C")

```

Loops

```

// declares i, checks i < myVariable, if it is, executes block and adds 1 to i
for (int i = 0; i < myVariable; i++) {
    System.out.println(i);
    if (i == 3) {
        break;
    }
}

```

```

}

// Keeps executing as long as the given condition is true
boolean under3 = true;
while (under3) {
    System.out.println(i);
    if (i == 3) {
        under3 = false;
    }
}

// Like while loop, but always executes once before checking condition
do {
    System.out.println(i);
    if (i == 3) {
        under3 = false;
    }
}
while (under3)

```

Fenceposting

To print a fencepost of a given length, we need some extra work.

```

static void fenceBuilder(int length) {
    for (i = 0; i < length; i++) {
        System.out.print("|--");
    }
}
// For fenceBuilder(4) this would print |--|--|--|-- which is missing its last
// post.

static void betterFenceBuilder(int length) {
    for (i = 0; i < length; i++) {
        System.out.print("|--");
    }
    System.out.print("|");
}
// For betterFenceBuilder(4) this would print |--|--|--|--|

```

There are many way to solve fenceposting problems, but usually you just need to make some redundant code outside the loop

Short-circuit evaluation

Short-circuit evaluation makes checking possibly illegal things possible.

Here, we want to check that the value in our array at a given index is greater than 4. We cannot get an `IndexOutOfBoundsException`, because checking the length always happens first, and at the first failure in an `and` statement, Java just gives up.

```
int[] myIntArr = new int[] {1, 2, 3, 4, 5, 6, 7};

static void valueChecker(int index) {
    if (index > myIntArr.length && myIntArr[index] > 4) {
        System.out.println("Wow!");
    }
}
```

Likewise, using the or || it terminates as soon as a true value is found

Classes

```
class myClass {
    // Fields
    private int myIntField;
    private String myStringField;
    // Public field, can be directly modified by client code, should generally
    be avoided.
    public double myDoubleField;
    // Fields may have default values
    private boolean foo = false;

    //Constructor, must be public
    public myClass(int myIntField, String myStringField) {
        // 'This' keyword refers to the field of the instance, when another
        variable has the same name.
        this.myIntField = myIntField;
        this.myStringField = myStringField;
        // Only use the this. keyword when it makes sense
        myDoubleField = 0.0;
    }

    // ToString method, allows for printing meaningful data
    @Override
    public String toString() {
        return Integer.toString(myIntField) + ":" + myStringField;
    }
}
```

Static

The static keyword indicates that for a given class, this will not change. All instances will have the same version of this. If one instance updates a static value, it is updated for all instances of the class.

Access rules

```
public class myClass{
```

```
    public String myString; // Can be accessed by everything
    protected int myInt; // Can be accessed by files in the same package, and
subclasses
    double myDouble; // Can be accessed by files in the same package
    private boolean myBool; // Can only be accessed by this class
}
```

Inheritance

```
public class Car {
    private String brand;
    private String model;
    private long price;

    public String getBrand() {
        return this.brand;
    }

    public String getModel() {
        return this.model;
    }

    public long getPrice() {
        return this.price;
    }
}
```

// Note, must be in separate files.

```
public class UsedCar extends Car {
    private int milesDriven;
    private Date lastServiced;
    private int previousOwners;

    public int getMilesDriven() {
        return this.milesDriven;
    }

    public Date getLastServiced() {
        return this.lastServiced;
    }

    public int getPreviousOwners() {
        return this.previousOwners;
    }

    // Overrides the method in the superclass
    @Override
    public long getPrice() {
        // super.getPrice allows you to access overwritten methods.
        return super.getPrice -
    }
```

```
((this.getMilesDriven()/1000)*(Math.pow(this.getPreviousOwners,2)));  
    }  
}
```

Superclass constructors

Subclasses can use their superclasses constructor and add some of their own.

```
public class One {  
    private int number;  
    private String name;  
  
    public One(number, name) {  
        this.number = number  
        this.name = name  
    }  
}  
  
public class Two extends One {  
    private String address;  
  
    public Two(number, name, address) {  
        super(number, name);  
        this.address = address;  
    }  
}
```

Overloading

Overloading allows for multiple constructors, where default values for the other fields can then be chosen.

```
public class PassedCourses {  
    boolean math;  
    boolean english;  
    boolean music;  
  
    public PassedCourses(boolean math, boolean english, boolean music) {  
        this.math = math;  
        this.english = english;  
        this.music = music;  
    }  
  
    public PassedCourses(boolean math) {  
        this(true, false, false)  
    }  
}
```

Exceptions

Giving a method the permission to throw exceptions can be required when working with files, where the compiler knows errors are inevitable.

```
public class myClass {
    // Adding the "throws Exception", the method is allowed to throw an
    exception
    static void UnknownError() throws Exception{
        throw new Exception("Unknown error occurred");
    }

    static void SpecificError(String a) throws IllegalArgumentException{
        if (a != 5) {
            throw new IllegalArgumentException("You gave the wrong input
            fool!");
        }
    }

    // Try/Catch methods allow graceful handling of errors.
    static int getIntegerInput(Scanner scanner) {
        while (true) {
            try {
                return scanner.nextInt();
            } catch (Exception e) {
                scanner.nextLine();
                System.out.println("Wrong input!");
            }
        }
    }
}
```

Scanners

Scanners turn a piece of text into tokens, usually delimited by spaces, tabs, newlines, and once a token has been read it gets thrown out.

Scanners have the `scanner.nextVariable` methods to return the next token as that type. If the next token the scanner is going over is not of the type requested, it will throw an exception.

The method `scanner.hasNextVariable` methods check if the next token does fit the variable type.

```
.nextInt() // int
.nextDouble() // double
.next() // String
.nextLine() // Entire line, String
```

```
import java.util.*;

Scanner consoleScanner = new Scanner(System.in);

if (consoleScanner.hasNextInt()) {
    int x = consoleScanner.nextInt();
} else {
    consoleScanner.nextLine(); // throw away the current line to go to next
    System.out.println("You have to write a real integer!");
}
```

Files

Note, when working with files, you either need `throws FileNotFoundException` in the method or need to wrap it in `try/catch`

Common methods:

```
import java.io.*; // Needs the io package

File foo = new File(bar.baz); // Makes object foo that represents the file
bar.baz

foo.exists(); // true/false if the file exists

foo.canRead(); // true/false if the file can be read

foo.isDirectory(); // true/false if the file is a directory

foo.isFile(); // true/false if the file is a file

foo.length(); // long integer for the size in bytes

foo.getAbsolutePath(); // returns absolute path including filename C:/path/to/
file/bar.baz

foo.delete(); // deletes the actual file

foo.getName(); // returns String of the files name without path, bar.baz

foo.renameTo(otherFile) // renames foo to the name that otherFile has, that
means it does NOT rename the file bar.baz to "otherFile", but instead renames
bar.baz to whatever the file that otherFile represents is named.
```

Example usage

```
import java.io.File;
import java.io.PrintStream;
```

```

import java.util.scanner;

public class fileStuffs {
    public static void main(String[] args) {
        File myFile = new File("FileLocation.txt"); // Just creates path in
memory, no exceptions

        // Reading from file
        try {
            Scanner fileContents = new Scanner(myfile);
        } catch (FileNotFoundException e) {
            System.out.println("The file was not found");
        }

        // Writing to files
        // If the file does not exist, it is created
        try {
            // Note, the file is wiped when first loaded.
            PrintStream myPrintStream = new PrintStream(myFile);
            myPrintStream.print("Hello");
            myPrintStream.println(" World!");

            myPrintStream.close();
        } catch (FileNotFoundException e) {
            System.out.println("The file was not found");
        }

        // printstream empties the file first
        close it agian
    }
}

```

Random

```

// Using java.util.Random
import java.util.Random

Random randGen = new Random();

int a = randGen.nextInt(43); // Number in range [0, 42]
double b = randGen.nextDouble(); // Number in range [0.0, 1.0[
boolean c = randGen.nextBoolean(); // True/false
double d = randGen.nextGaussian(); // Normal distribution, mean=0, standard
deviation=1

// Using Math.random
double random = Math.random(); // Number in range [0.0, 1.0[

```

Arrays

On primitive types, unset values are `0` or `false` whereas objects get `null`

`myArray.length` returns the length.

`myArray[i]` gives the value of the *i*'th object, 0-indexed.

```
// Declaring empty int array named "myIntArr" with length 4
int[] myIntArr = new int[4];

// Declaring an array containing 3 integer arrays, each containing 5 integers.
int[][] myIntArrArr = new int[3][5];

// Declaring array with preset values
String[] names = {"John", "Adam", "Afton"};

// Declaring array of arrays with preset values
String[][] names = {{ "Michael", "Aton"}, {"William", "Afton"}}
```

Changeability

If a gets assigned to be equal to array b, and a then gets changed, b will also get changed.

```
String[] names = {"Correct", "Adam", "Afton"};
String[] foo = names;
names[0] = "Wrong";
System.out.println(foo[0]);
// prints "Wrong"
```

To be able to get an independent copy, use `Arrays.copyOf` or `.clone()`

```
String[] names = {"Correct", "Adam", "Afton"};
String[] foo = Arrays.copyOf(names, names.length)
String[] bar = names.clone()
names[0] = "Wrong";

System.out.println(foo[0]);
// prints "Correct"

System.out.println(bar[0]);
// prints "Correct"
```

`Arrays.copyOf` can also be used to increase the size of the array when copying. (can both be shorter or longer).

Note that this is only a *shallow copy*, so if it is an array of objects, and one of the underlying objects gets

changed in one, it the object is still changed in the other.

ForEach

ForEach loops allow simpler loops, but dont allow you to use the `i` index variable.

```
int[] myArr = {1,2,3,4,5,6,7}

for (in n : myArr) {
    System.out.print(n);
}
for (int i = 0; i < myArr.length; i++) {
    System.out.print(myArr[i]);
}
```

Note that the objects used in a ForEach loop is a copy, so changing it will not work:

```
for (int n : myArr) {
    n += 1;
}
// This does NOT work
```

Superclass Arrays

Arrays can be made containing any object. If a class is a subclass of a superclass, it can be added to arrays of that superclass.

If you have class `Car` and subclass `UsedCar` any usedcar objects can also be put in the car array.

Likewise, any class implementing the interface `HasWheels` is able to be added to a `HasWheels[]` array.

Other

To string: `Arrays.toString(arr)`

To string multi-dimensional: `Arrays.deepToString(myArr)`

Compare: `Arrays.equals(arr1, arr2);`

Fill: `Arrays.fill(arr, value)`

Sort: `Arrays.sort(arr)`

lambda function

In `Function<Integer, String>` the first denotes input, the second denotes output.

In `(x -> x + 32)` we denote that value `x` gets turned into `x+32`

```
import java.util.function.Function;

static void lambdaTest() {
    Function<Integer, Integer> func = (x -> x + 32);
    System.out.println(func.apply(32));
}
```

Enums

Enums are a custom variable type with a set list of possible values. Written in their own file, typically in caps.

```
public enum Suit {
    CLUBS, DIAMONDS, HEARTS, SPADES
}

// In client code
Suit myCardSuit = Suit.DIAMONDS;
```

JavaDoc

JavaDoc are comments written either above the class, methods, fields and more.

```
/**
 * My super cool class
 * @author John Doe
 */

public class myClass {
    private int num;

    /**
     * Adds an integer to num
     * The number must be greater than or equal to 4
     *
     * @param a the number to be added
     * @return returns boolean for if the number exceeds 49
     * @exception IllegalArgumentException
     */

    public boolean addNumber(int a) {
        if (a<4) {
            throw new IllegalArgumentException("Incompatible number " + a);
        }
        num += a;

        return num>49;
    }
}
```

```
}  
}
```

Creating a Jar file

In the folder with the classes, create file `manifest.mf` with contents:

```
Manifest-Version: 1.0  
Main-Class: <main-class-name>
```

In the terminal write `jar cvmf manifest.mf <jar-filename>.jar *.class`

ASCII table

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
32	20		48	30	0	65	41	A	97	61	a
33	21	!	49	31	1	66	42	B	98	62	b
34	22	"	50	32	2	67	43	C	99	63	c
35	23	#	51	33	3	68	44	D	100	64	d
36	24	\$	52	34	4	69	45	E	101	65	e
37	25	%	53	35	5	70	46	F	102	66	f
38	26	&	54	36	6	71	47	G	103	67	g
39	27	'	55	37	7	72	48	H	104	68	h
40	28	(	56	38	8	73	49	I	105	69	i
41	29	)	57	39	9	74	4A	J	106	6A	j
42	2A	*				75	4B	K	107	6B	k
43	2B	+				76	4C	L	108	6C	l
44	2C	,				77	4D	M	109	6D	m
45	2D	-				78	4E	N	110	6E	n
46	2E	.				79	4F	O	111	6F	o
47	2F	/				80	50	P	112	70	p
58	3A	:				81	51	Q	113	71	q
59	3B	;				82	52	R	114	72	r
60	3C	<				83	53	S	115	73	s

